Ledline

Generated by Doxygen 1.9.4

1 Todo List 1

2 Hierarchical Index 3

2.1 Class Hierarchy . . . 3

3 Class Index 5

3.1 Class List . . . 5

4 File Index 7

4.1 File List . . . 7

5 Class Documentation 9

5.1 begin Class Reference . . . 9

5.1.1 Detailed Description . . . 10

5.1.2 Member Function Documentation . . . 10

5.1.2.1 trace() . . . 10

5.2 contact_bounce_meter Class Reference . . . 11

5.2.1 Detailed Description . . . 12

5.2.2 Constructor & Destructor Documentation . . . 12

5.2.2.1 contact_bounce_meter() . . . 12

5.2.3 Member Function Documentation . . . 12

5.2.3.1 begin() . . . 13

5.2.3.2 CAPT() . . . 13

5.2.3.3 clearTCNT1() . . . 13

5.2.3.4 disablePCINT0() . . . 13

5.2.3.5 disableTIMER1_OVF_CAPT() . . . 13

5.2.3.6 enablePCINT0() . . . 13

5.2.3.7 enableTIMER1_OVF_CAPT() . . . 14

5.2.3.8 operator()() . . . 14

5.2.3.9 OVF() . . . 14

5.2.3.10 PCINT() . . . 14

5.2.4 Member Data Documentation . . . 14

5.2.4.1 Self . . . 14

5.3 ledline::dimm Class Reference . . . 15

5.3.1 Detailed Description . . . 17

5.3.2 Member Function Documentation . . . 17

5.3.2.1 next() . . . 17

5.3.2.2 operator()() . . . 17

5.3.2.3 prev() . . . 17

5.3.2.4 set() . . . 17

5.4 ir_remote_control::input Class Reference . . . 18

5.4.1 Detailed Description . . . 19

5.4.2 Member Function Documentation . . . 19

5.4.2.1 trace() . . . 19

5.5 ir_remote_control Class Reference . . . 19
5.5.1 Detailed Description . . . 19
5.6 ledline Class Reference . . . 20
5.6.1 Detailed Description . . . 21
5.6.2 Member Typedef Documentation . . . 21
5.6.2.1 brightness . . . 21
5.6.3 Member Enumeration Documentation . . . 21
5.6.3.1 preset . . . 21
5.6.4 Member Function Documentation . . . 22
5.6.4.1 gamma() . . . 22
5.6.4.2 mount() . . . 30
5.6.4.3 operator uint8_t() . . . 30
5.6.4.4 operator++() . . . 31
5.6.4.5 operator--() . . . 31
5.6.4.6 operator=() . . . 31
5.6.4.7 operator[]() . . . 31
5.7 ir_remote_control::listen Class Reference . . . 32
5.7.1 Detailed Description . . . 33
5.7.2 Member Function Documentation . . . 33
5.7.2.1 ISRoutine() . . . 33
5.7.2.2 operator()() . . . 33
5.8 task::queue Class Reference . . . 33
5.8.1 Detailed Description . . . 34
5.8.2 Constructor & Destructor Documentation . . . 34
5.8.2.1 queue() . . . 34
5.8.3 Member Function Documentation . . . 34
5.8.3.1 get() . . . 34
5.8.3.2 put() . . . 34
5.9 task::scheduler Class Reference . . . 35
5.9.1 Detailed Description . . . 36
5.9.2 Member Function Documentation . . . 36
5.9.2.1 execute() . . . 36
5.9.3 Member Data Documentation . . . 36
5.9.3.1 Self . . . 36
5.10 ledline::dimm::stat Struct Reference . . . 37
5.10.1 Member Function Documentation . . . 38
5.10.1.1 print() . . . 38
5.10.1.2 trace() . . . 38
5.11 ledline::task Class Reference . . . 38
5.11.1 Detailed Description . . . 39
5.12 task Class Reference . . . 40
5.12.1 Detailed Description . . . 40

5.12.2 Member Function Documentation . . . 40
5.12.2.1 operator()() . . . 41
5.13 test Class Reference . . . 41
5.13.1 Detailed Description . . . 42
5.13.2 Member Function Documentation . . . 42
5.13.2.1 trace() . . . 42
5.14 traceable Class Reference . . . 43
5.14.1 Detailed Description . . . 43
5.14.2 Member Function Documentation . . . 44
5.14.2.1 operator()() . . . 44
5.14.2.2 trace() . . . 44
5.15 udiv8_t Struct Reference . . . 44

6 File Documentation **45**
6.1 contact_bounce_meter.h File Reference . . . 45
6.1.1 Detailed Description . . . 46
6.1.1.1 Measurements . . . 46
6.1.2 Function Documentation . . . 51
6.1.2.1 ISR() [1/3] . . . 51
6.1.2.2 ISR() [2/3] . . . 51
6.1.2.3 ISR() [3/3] . . . 51
6.2 contact_bounce_meter.h . . . 51
6.3 Ledline.ino File Reference . . . 53
6.3.1 Detailed Description . . . 55
6.3.2 Macro Definition Documentation . . . 55
6.3.2.1 PIN_BOUNCE . . . 55
6.3.2.2 PIN_TOGGLE . . . 55
6.3.3 Function Documentation . . . 55
6.3.3.1 ISR() . . . 56
6.4 task.h . . . 56
6.5 udiv.h . . . 58

Index **59**

Chapter 1

Todo List

Member contact_bounce_meter::CAPT ()

Reset IdleCounter after latest measurement. Now idle overflows is countet after the first measure - an error.

Member ISR (PCINT2_vect)

This interrupt is to be used for toggling light from external switch.

Member PIN_TOGGLE

Connect simple one-way switch instead of test button to toggle light to pin. On changing switch state ledline is toggling between min and max levels. It may be used by usual wall light switch to toggle ledline without remote control.

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

- ir_remote_control . . . 19
- ledline . . . 20
- task::queue . . . 33
- task::scheduler . . . 35
- task . . . 40
  - contact_bounce_meter . . . 11
  - ir_remote_control::listen . . . 32
  - ledline::task . . . 38
    - ledline::dimm . . . 15
  - traceable . . . 43
    - begin . . . 9
    - ir_remote_control::input . . . 18
    - ledline::dimm::stat . . . 37
    - test . . . 41
- udiv8_t . . . 44

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

begin 9
contact_bounce_meter 11
ledline::dimm 15
ir_remote_control::input 18
ir_remote_control 19
ledline 20
ir_remote_control::listen 32
task::queue 33
task::scheduler 35
ledline::dimm::stat 37
ledline::task 38
task 40
test 41
traceable 43
udiv8_t 44

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

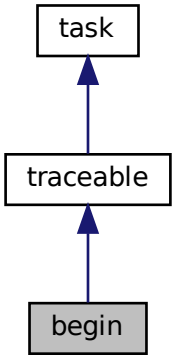
contact_bounce_meter.h 45
Ledline.ino 53
task.h . . . . . ??
udiv.h . . . . . ??

Chapter 5

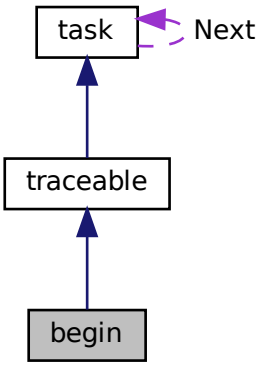
Class Documentation

5.1 begin Class Reference

Inheritance diagram for begin:



Collaboration diagram for begin:



Static Public Member Functions

- static void **invoke** ()

Private Member Functions

- void trace () noexcept
- void **version** ()

Additional Inherited Members

5.1.1 Detailed Description

Entry point.

5.1.2 Member Function Documentation

5.1.2.1 trace()

```
void begin::trace ( )  [inline], [private], [virtual], [noexcept]
```

Defines traceable routine.

Implements traceable.

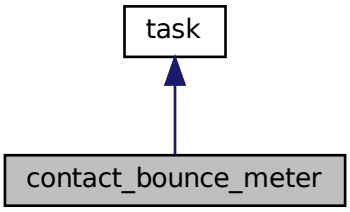
The documentation for this class was generated from the following file:

- Ledline.ino

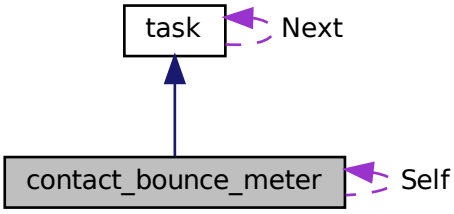
5.2 contact_bounce_meter Class Reference

```
#include <contact_bounce_meter.h>
```

Inheritance diagram for contact_bounce_meter:



Collaboration diagram for contact_bounce_meter:



Public Member Functions

- contact_bounce_meter ()
- void begin ()
- void PCINT ()
- void OVF ()
- void CAPT ()

Static Public Attributes

- static contact_bounce_meter Self

Private Member Functions

- void operator() ()
- void enablePCINT0 ()
- void disablePCINT0 ()
- void enableTIMER1_OVF_CAPT ()
- void disableTIMER1_OVF_CAPT ()
- void clearTCNT1 ()

Private Attributes

- uint16_t **Capture**

  ICR1 value saved in ICI ISR.
- uint8_t **Offset**

  TOV counter value saved at ICI ISR.
- uint8_t **Counter**

  TOV overflow counter.
- bool **Idle**

  Idle flag is used to check if any ICI has been occurs since previous TOV.
- uint8_t **IdleCounter**

  Number of sequential idle TOV intervals.

Static Private Attributes

- static const uint8_t **MaxIdleCounter** = 2

5.2.1 Detailed Description

Contact bounce meter.

5.2.2 Constructor & Destructor Documentation

5.2.2.1 contact_bounce_meter()

```
contact_bounce_meter::contact_bounce_meter ( ) [inline]
```

Create meter in idle state.

5.2.3 Member Function Documentation

5.2.3.1 begin()

```
void contact_bounce_meter::begin ( ) [inline]
```

Begin measuring process.

Unconditionally enables pin change interrupt 0 to start measuring process on PCINT0 pin.

5.2.3.2 CAPT()

```
void contact_bounce_meter::CAPT ( ) [inline]
```

TIMER1_CAPT interrupt.

Save captured and overflow counter values. Reset idle flat to continue measuring.

Todo Reset IdleCounter after latest measurement. Now idle overflows is countet after the first measure - an error.

5.2.3.3 clearTCNT1()

```
void contact_bounce_meter::clearTCNT1 ( ) [inline], [private]
```

Clear TCNT1.

5.2.3.4 disablePCINT0()

```
void contact_bounce_meter::disablePCINT0 ( ) [inline], [private]
```

Disable PCINT0.

5.2.3.5 disableTIMER1_OVF_CAPT()

```
void contact_bounce_meter::disableTIMER1_OVF_CAPT ( ) [inline], [private]
```

Disable TIMER1 OVF and CAPT.

5.2.3.6 enablePCINT0()

```
void contact_bounce_meter::enablePCINT0 ( ) [inline], [private]
```

Enable PCINT0.

5.2.3.7 enableTIMER1_OVF_CAPT()

```
void contact_bounce_meter::enableTIMER1_OVF_CAPT ( ) [inline], [private]
```

Enable TIMER1 OVF and CAPT.

5.2.3.8 operator()()

```
void contact_bounce_meter::operator() ( ) [inline], [private], [virtual]
```

Output measurement results.

Implements task.

5.2.3.9 OVF()

```
void contact_bounce_meter::OVF ( ) [inline]
```

TIMER1_OVF interrupt.

Increases overflow counter and stops measuring process if number of sequential idle overflow intervals exceed limit.

5.2.3.10 PCINT()

```
void contact_bounce_meter::PCINT ( ) [inline]
```

ISR to start measuring process.

5.2.4 Member Data Documentation

5.2.4.1 Self

```
contact_bounce_meter contact_bounce_meter::Self [static]
```

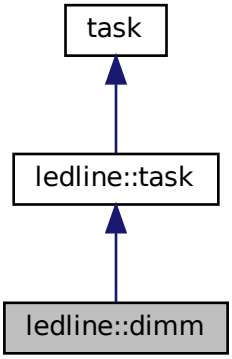
Singleton object.

The documentation for this class was generated from the following file:

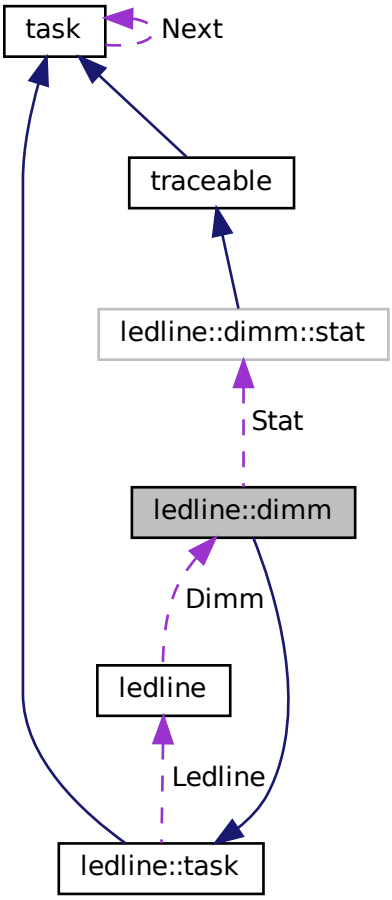
- contact_bounce_meter.h

5.3 ledline::dimm Class Reference

Inheritance diagram for ledline::dimm:



Collaboration diagram for ledline::dimm:



Classes

- struct stat

Public Member Functions

- **dimm** (ledline &_ledline)
- void operator() ()
- void **invoke** (uint8_t _level)
- void **toggle** ()

  Toggle ledline.
- void set (preset _mode)
- preset next ()
- preset prev ()

Private Attributes

- int **Level**
- int **Distance**
- unsigned long **Time**
- ledline::dimm::stat **Stat**

Static Private Attributes

- static const int **LevelingTime** = 1024

Additional Inherited Members

5.3.1 Detailed Description

Dimming task.

5.3.2 Member Function Documentation

5.3.2.1 next()

```
preset ledline::dimm::next ( ) [inline]
```

Switch to the next higher preset brightness level.

Returns

Next higher preset level.

5.3.2.2 operator()()

```
void ledline::dimm::operator() ( ) [inline], [virtual]
```

Implements task.

5.3.2.3 prev()

```
preset ledline::dimm::prev ( ) [inline]
```

Switch to the previous lower preset briughtness level.

Returns

Previous lower preset mode.

5.3.2.4 set()

```
void ledline::dimm::set (
            preset _mode ) [inline]
```

Dimm to preset mode brightness.

Parameters

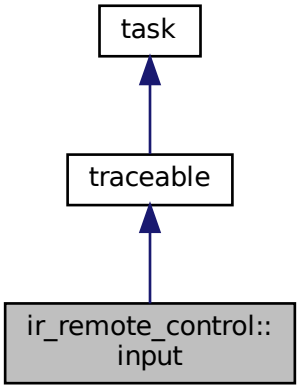
in	*_mode*	Preset mode.

The documentation for this class was generated from the following file:

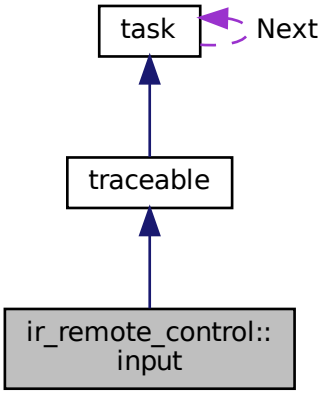
- Ledline.ino

5.4 ir_remote_control::input Class Reference

Inheritance diagram for ir_remote_control::input:



Collaboration diagram for ir_remote_control::input:



Private Member Functions

- void trace () noexcept

Additional Inherited Members

5.4.1 Detailed Description

Read control data. Iterative task which tests IR control until meaningfull data occurs.

5.4.2 Member Function Documentation

5.4.2.1 trace()

```
void ir_remote_control::input::trace ( )  [inline], [private], [virtual], [noexcept]
```

Defines traceable routine.

Implements traceable.

The documentation for this class was generated from the following file:

- Ledline.ino

5.5 ir_remote_control Class Reference

Classes

- class input
- class listen

5.5.1 Detailed Description

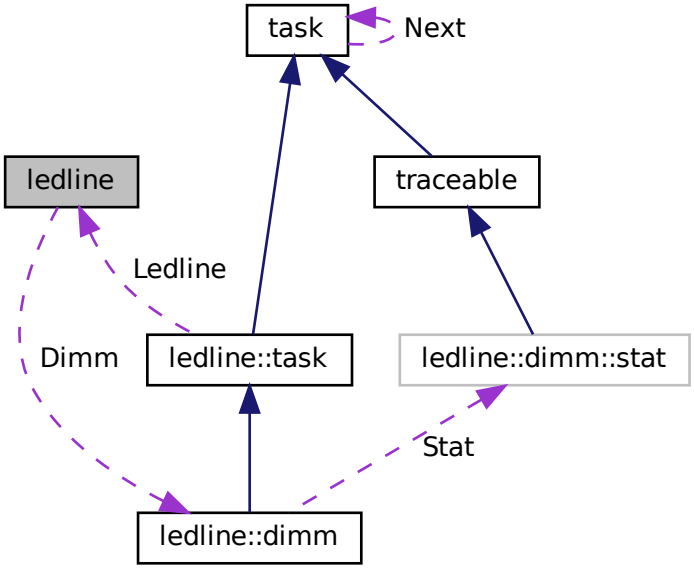
IR remote control.

The documentation for this class was generated from the following file:

- Ledline.ino

5.6 ledline Class Reference

Collaboration diagram for ledline:



Classes

- class dimm
- class task

Public Types

- enum class preset {
  No , Night , Twilight , Day ,
  Bound }
- typedef uint8_t brightness

Public Member Functions

- void mount (uint8_t _pin)
- operator uint8_t () const
- ledline & operator= (uint8_t _level)
- uint8_t operator++ ()
- uint8_t operator-- ()
- brightness & operator[ ] (preset _mode)

Static Public Member Functions

- static uint8_t gamma (uint8_t _z, uint8_t _r=0)

Public Attributes

- ledline::dimm **Dimm**

Static Public Attributes

- static const uint8_t **MinLevel** = 0x00

  Minimum brightness level.
- static const uint8_t **MaxLevel** = 0xFF

  Maximum brightness level.

Private Attributes

- uint8_t **Level**
- uint8_t **Pin**
- brightness **Preset** [(uint8_t) preset::Bound]

  Preset brightness levels.

5.6.1 Detailed Description

Ledline.

5.6.2 Member Typedef Documentation

5.6.2.1 brightness

```
typedef uint8_t ledline::brightness
```

Brightness level.

5.6.3 Member Enumeration Documentation

5.6.3.1 preset

```
enum class ledline::preset  [strong]
```

Preset brightness.

Enumerator

No	No illumination.
Night	To find a way.
Twilight	To see everythin in general.
Day	To read.
Bound	Upper bound.

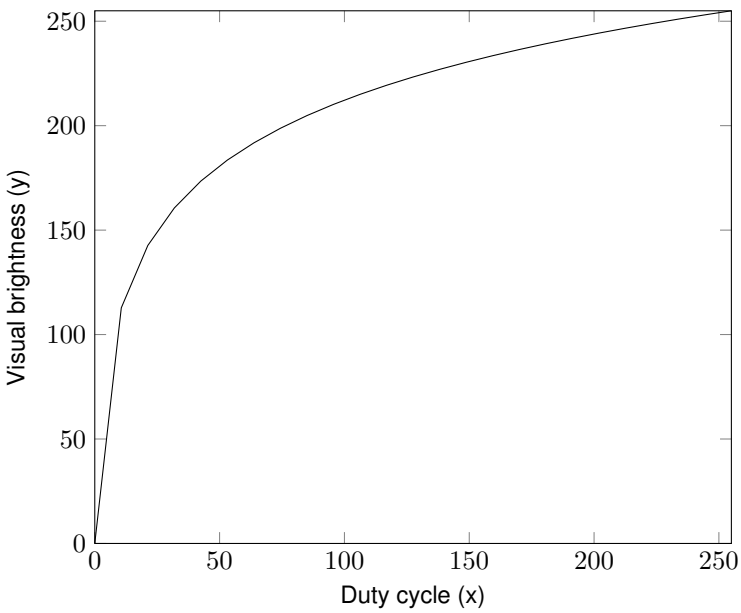
5.6.4 Member Function Documentation

5.6.4.1 gamma()

```
static uint8_t ledline::gamma (
            uint8_t _z,
            uint8_t _r = 0 )  [inline], [static]
```

Brightness correction (gamma correction).

It is observed that visual brightness of the led line has no linear dependence on PWM duty cycle. While duty cycle value runs from 0 up to 255, visual brightness increases faster on lower values and slower for higher values of duty cycle. This makes to suppose logarithmic dependence between duty cycle and its visual brightness something as a follows



This curve may be represented by binary logarithmic function, shifted and scaled to be defined on domain [0,255] with the same values range

$$y = f(x) = k \log_2(x + 1)$$

Scaling factor can be expressed from the following equation

$$255 = k \log_2(255 + 1)$$

So supposed brightness function can be expressed as

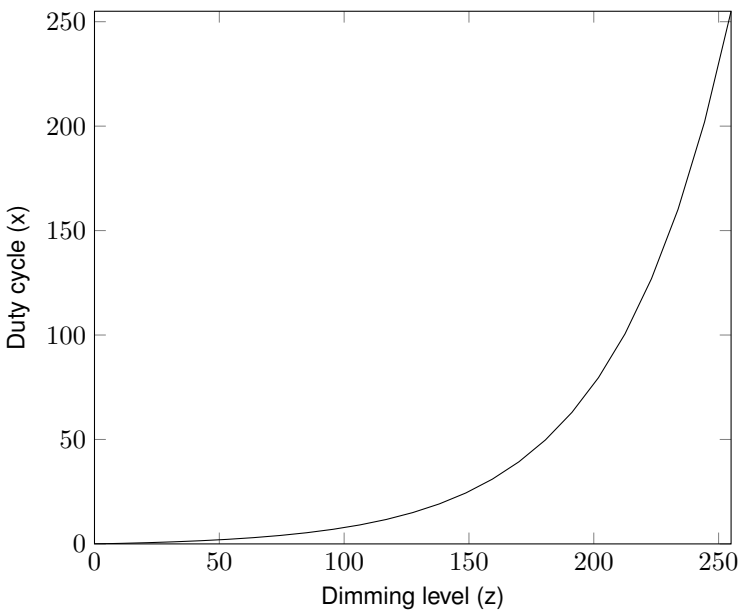
$$y = f(x) = \frac{255}{\log_2(255+1)} \log_2(x+1) = \frac{255}{8} \log_2(x+1)$$

The main goail is to acheve linear dependence of visual brightness on dimming level - integer control parameter of brightness in the range from 0 to 255. For this gamma correction function $x = g(z)$ should be provided to convert dimming level to duty cycle so that $y = f(g(z)) = z$, where z - is a dimming level and y - visual brightness. Obviously such a gamma correction should be an inverse function of brightness function

$$y = z = f(g(z)) = \frac{255}{8} \log_2(g(z)+1) \Rightarrow \frac{8}{255} z = \log_2(g(z)+1) \Rightarrow 2^{\frac{8}{255}z} = g(z) + 1$$

Thus the gamma correction function will be following

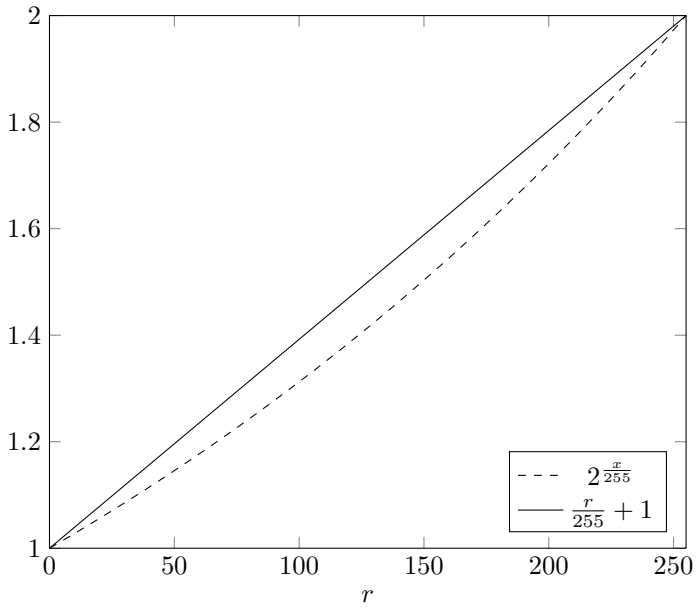
$$x = g(z) = 2^{\frac{8}{255}z} - 1$$



Whereas integer power of two can be calculated effectivly as a bitwise shift then its picewise linear approximation can be used be used to calclulate gamma correction without floating point arithmetic. $8z$ can be represented as $255m + r$, where $m = \lfloor 8z/255 \rfloor$ and $r = 8z \pmod{255}$. Then

$$g(z) = 2^{\frac{8z}{255}} - 1 = 2^{\frac{255m+r}{255}} - 1 = 2^m * 2^{\frac{r}{255}} - 1$$

Integer part 2^m of gamma correction function can be calculated directly by bitwise shift operation. Fractional part $2^{r/255}$ can be subsituted by linear approximation $r/255 + 1$.



Thus picewise linearly approximated gamma correction will be as follows

$$g(z) = 2^m \left(\frac{r}{255}+1\right) - 1 = 2^{\lfloor 8z/255 \rfloor} \left(\frac{8z \pmod{255}}{255}+1\right) - 1 = \frac{2^{\lfloor 8z/255 \rfloor}(8z \pmod{255}+255)}{255} - 1$$

Or

$$g(z) = \frac{2^{\lfloor 8z/255 \rfloor}(8z \pmod{255}+255)}{255} - 1$$

Whereas integer division provides truncated result (rounded toward zero) remainder may be analyzed to increase precision. Let

$$v(z) = 2^{\lfloor 8z/255 \rfloor}(8z \pmod{255}+255)$$

Then

$$g(z) = \lfloor \frac{v(z)}{255} \rfloor - 1 + \begin{cases} 1 & v(z) \pmod{255} > n \\ 0 & \text{otherwise} \end{cases}$$

Here n is a rounding mode parameter. If $n = 0$ then $g(z)$ is rounded up, if $n = 255$ then $g(z)$ is rounded down and any value $0 < n < 255$ means intermediate round up mode - for example $n = 127$ may be considered as a rounding half up mode. The following table shows complete duty cycle calculation for different n.

Level	n=255	n=127	n=0
0	0	0	0
1	0	0	1
2	0	0	1
3	0	0	1
4	0	0	1
5	0	0	1

Level	n=255	n=127	n=0
6	0	0	1
7	0	0	1
8	0	0	1
9	0	0	1
10	0	0	1
11	0	0	1
12	0	0	1
13	0	0	1
14	0	0	1
15	0	0	1
16	0	1	1
17	0	1	1
18	0	1	1
19	0	1	1
20	0	1	1
21	0	1	1
22	0	1	1
23	0	1	1
24	0	1	1
25	0	1	1
26	0	1	1
27	0	1	1
28	0	1	1
29	0	1	1
30	0	1	1
31	0	1	1
32	1	1	2
33	1	1	2
34	1	1	2
35	1	1	2
36	1	1	2
37	1	1	2
38	1	1	2
39	1	1	2
40	1	2	2
41	1	2	2
42	1	2	2
43	1	2	2
44	1	2	2
45	1	2	2
46	1	2	2
47	1	2	2
48	3	2	3
49	3	2	3
50	3	2	3
51	3	2	3
52	3	2	3
53	3	2	3
54	3	2	3
55	3	2	3
56	3	3	3
57	3	3	3
58	3	3	3

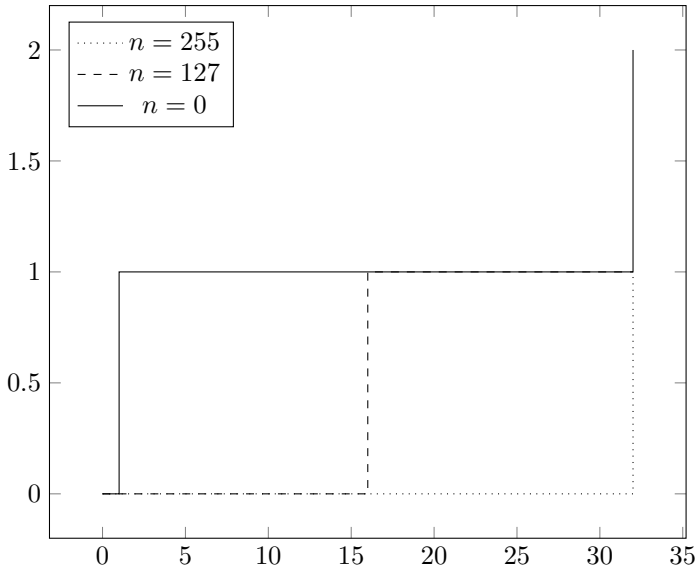
Level	n=255	n=127	n=0
59	3	3	3
60	3	3	3
61	3	3	3
62	3	3	3
63	3	3	3
64	3	3	4
65	3	3	4
66	3	3	4
67	3	3	4
68	3	4	4
69	3	4	4
70	3	4	4
71	3	4	4
72	5	4	5
73	5	4	5
74	5	4	5
75	5	4	5
76	5	5	5
77	5	5	5
78	5	5	5
79	5	5	5
80	5	5	6
81	5	5	6
82	5	5	6
83	5	5	6
84	5	6	6
85	5	6	6
86	5	6	6
87	5	6	6
88	7	6	7
89	7	6	7
90	7	6	7
91	7	6	7
92	7	7	7
93	7	7	7
94	7	7	7
95	7	7	7
96	7	7	8
97	7	7	8
98	7	8	8
99	7	8	8
100	9	8	9
101	9	8	9
102	9	9	9
103	9	9	9
104	9	9	10
105	9	9	10
106	9	10	10
107	9	10	10
108	11	10	11
109	11	10	11
110	11	11	11
111	11	11	11

Level	n=255	n=127	n=0
112	11	11	12
113	11	11	12
114	11	12	12
115	11	12	12
116	13	12	13
117	13	12	13
118	13	13	13
119	13	13	13
120	13	13	14
121	13	13	14
122	13	14	14
123	13	14	14
124	15	14	15
125	15	14	15
126	15	15	15
127	15	15	15
128	15	15	16
129	15	16	16
130	17	16	17
131	17	17	17
132	17	17	18
133	17	18	18
134	19	18	19
135	19	19	19
136	19	19	20
137	19	20	20
138	21	20	21
139	21	21	21
140	21	21	22
141	21	22	22
142	23	22	23
143	23	23	23
144	23	23	24
145	23	24	24
146	25	24	25
147	25	25	25
148	25	25	26
149	25	26	26
150	27	26	27
151	27	27	27
152	27	27	28
153	27	28	28
154	29	28	29
155	29	29	29
156	29	29	30
157	29	30	30
158	31	30	31
159	31	31	31
160	31	32	32
161	33	33	33
162	33	34	34
163	35	35	35
164	35	36	36

Level	n=255	n=127	n=0
165	37	37	37
166	37	38	38
167	39	39	39
168	39	40	40
169	41	41	41
170	41	42	42
171	43	43	43
172	43	44	44
173	45	45	45
174	45	46	46
175	47	47	47
176	47	48	48
177	49	49	49
178	49	50	50
179	51	51	51
180	51	52	52
181	53	53	53
182	53	54	54
183	55	55	55
184	55	56	56
185	57	57	57
186	57	58	58
187	59	59	59
188	59	60	60
189	61	61	61
190	61	62	62
191	63	63	63
192	65	65	65
193	67	67	67
194	69	69	69
195	71	71	71
196	73	73	73
197	75	75	75
198	77	77	77
199	79	79	79
200	81	81	81
201	83	83	83
202	85	85	85
203	87	87	87
204	89	89	89
205	91	91	91
206	93	93	93
207	95	95	95
208	97	97	97
209	99	99	99
210	101	101	101
211	103	103	103
212	105	105	105
213	107	107	107
214	109	109	109
215	111	111	111
216	113	113	113
217	115	115	115

Level	n=255	n=127	n=0
218	117	117	117
219	119	119	119
220	121	121	121
221	123	123	123
222	125	125	125
223	127	127	127
224	131	131	131
225	135	135	135
226	139	139	139
227	143	143	143
228	147	147	147
229	151	151	151
230	155	155	155
231	159	159	159
232	163	163	163
233	167	167	167
234	171	171	171
235	175	175	175
236	179	179	179
237	183	183	183
238	187	187	187
239	191	191	191
240	195	195	195
241	199	199	199
242	203	203	203
243	207	207	207
244	211	211	211
245	215	215	215
246	219	219	219
247	223	223	223
248	227	227	227
249	231	231	231
250	235	235	235
251	239	239	239
252	243	243	243
253	247	247	247
254	251	251	251
255	255	255	255

Rounding down and rounding half up mode will lead to zero brightness level (swiched off led line) in spite of positive dimming level near to its zero values (from 0 to 31 dimming level). It is not convinient when brightness is corrected manually (correction continues when led line is off). So more suitable is rounding up mode with $n = 0$. It makes non-zero duty cycle for all non-zero dimming levels. Here graphs of gamma function for differen n near to zero level.



Parameters

in	↩ _↩ *z*	Dimming level (linear dimming control).
in	↩ _↩ *r*	Rounding parameter

Returns

Corrected duty cycle corresponding to specified dimming level.

5.6.4.2 mount()

```
void ledline::mount (
            uint8_t _pin )  [inline]
```

Mount ledline on specified PWM pin.

5.6.4.3 operator uint8_t()

```
ledline::operator uint8_t ( ) const  [inline]
```

Get current brightness level.

Returns

Current brightness level.

5.6.4.4 operator++()

```
uint8_t ledline::operator++ ( ) [inline]
```

Increase current ledline brightness level by 1.

If current level becomes higher than MaxLevel it rounds to MinLevel.

5.6.4.5 operator--()

```
uint8_t ledline::operator-- ( ) [inline]
```

Decrese current brightness level by 1.

If current level becomes lower than MinLevel it rouns to MaxLevel. Brightness adjustment.

5.6.4.6 operator=()

```
ledline & ledline::operator= (
            uint8_t _level ) [inline]
```

Set current brightness level.

Parameters

in	*_level*	Brightness level.

Returns

Modified ledline object reference.

5.6.4.7 operator[]()

```
brightness & ledline::operator[] (
            preset _mode ) [inline]
```

Sets/gets preset mode brightness.

Parameters

in	*_mode*	Preset mode.

Returns

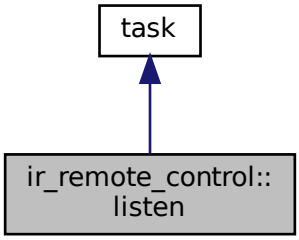
Preset mode brightness.

The documentation for this class was generated from the following file:

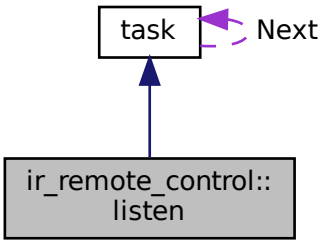
- Ledline.ino

5.7 ir_remote_control::listen Class Reference

Inheritance diagram for ir_remote_control::listen:



Collaboration diagram for ir_remote_control::listen:



Static Public Member Functions

- static void ISRoutine ()

Private Member Functions

- void operator() ()

Additional Inherited Members

5.7.1 Detailed Description

Start IrReceive listening from ISR.

5.7.2 Member Function Documentation

5.7.2.1 ISRoutine()

```
static void ir_remote_control::listen::ISRoutine ( )  [inline], [static]
```

ISR for IR control pin interrupt.

5.7.2.2 operator()()

```
void ir_remote_control::listen::operator() ( )  [inline], [private], [virtual]
```

Start listening from main process.

Implements task.

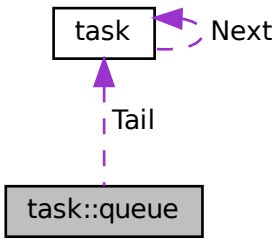
The documentation for this class was generated from the following file:

- Ledline.ino

5.8 task::queue Class Reference

```
#include <task.h>
```

Collaboration diagram for task::queue:



Public Member Functions

- queue ()
- task * **head** ()
- task * **tail** ()
- task * put (task *t)
- task * get ()

Private Attributes

- task * **Tail**

5.8.1 Detailed Description

Task queue.

5.8.2 Constructor & Destructor Documentation

5.8.2.1 queue()

```
task::queue::queue ( ) [inline]
```

Create empty queue.

5.8.3 Member Function Documentation

5.8.3.1 get()

```
task * task::queue::get ( ) [inline]
```

Removes head task from queue and returns pointer to it.

Does not check if queue is not empty and in the case result is unpredictable.

Returns

Pointer to removed task;

5.8.3.2 put()

```
task * task::queue::put (
            task * t ) [inline]
```

Put task in queue.

Puts task at the queue if it is not yet in any queue and returns pointer to the task. Does nothing and return NULL if task already in a queue.

Parameters

in	↩ _↩ *t*	Task to put into the queue.

Returns

Pointer to the task or NULL if task already is in queue.

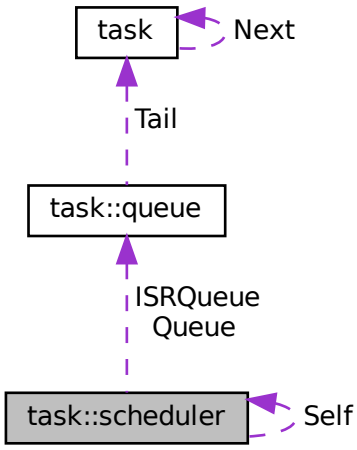
The documentation for this class was generated from the following file:

- task.h

5.9 task::scheduler Class Reference

```
#include <task.h>
```

Collaboration diagram for task::scheduler:



Public Member Functions

- task ∗ **put** (task ∗t)
- task ∗ **putISR** (task ∗t)
- void execute (bool _sleep=false, uint8_t _mode=SLEEP_MODE_IDLE)

  Executes next task from queue. Takes and executes a task from ISR queue or main queue. If both queues are empty and _sleep parameter is true puts device into sleep mode specified by parameter _mode.

Static Public Attributes

- static scheduler Self

Private Attributes

- queue **Queue**
- queue **ISRQueue**
- bool **ISRFlag**

5.9.1 Detailed Description

Task scheduler.

5.9.2 Member Function Documentation

5.9.2.1 execute()

```
void task::scheduler::execute (
            bool _sleep = false,
            uint8_t _mode = SLEEP_MODE_IDLE )  [inline]
```

Executes next task from queue. Takes and executes a task from ISR queue or main queue. If both queues are empty and _sleep parameter is true puts device into sleep mode specified by parameter _mode.

Parameters

in	*_sleep*	Put device into sleeping mode if ISR and main queues are empty.
in	*_mode*	Sleeping mode.

5.9.3 Member Data Documentation

5.9.3.1 Self

```
task::scheduler task::scheduler::Self  [static]
```

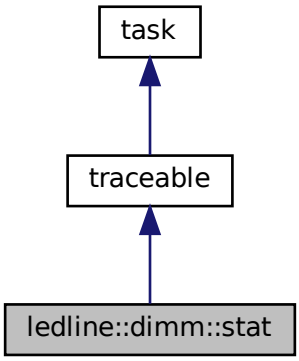
Task scheduler singleton object.

The documentation for this class was generated from the following files:

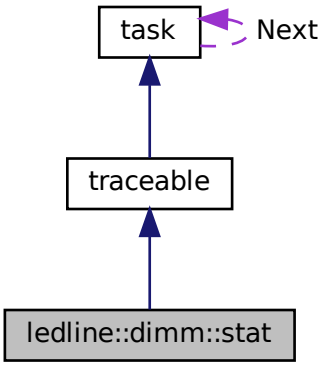
- task.h
- task.cpp

5.10 ledline::dimm::stat Struct Reference

Inheritance diagram for ledline::dimm::stat:



Collaboration diagram for ledline::dimm::stat:



Public Member Functions

- void trace () noexcept
- void print ()

Public Attributes

- uint16_t **Count**
- uint16_t **Time**

5.10.1 Member Function Documentation

5.10.1.1 print()

```
void ledline::dimm::stat::print ( ) [inline]
```

Print dimming statistics.

5.10.1.2 trace()

```
void ledline::dimm::stat::trace ( ) [inline], [virtual], [noexcept]
```

Defines traceable routine.

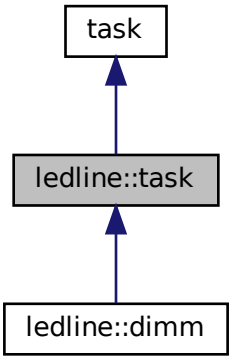
Implements traceable.

The documentation for this struct was generated from the following file:

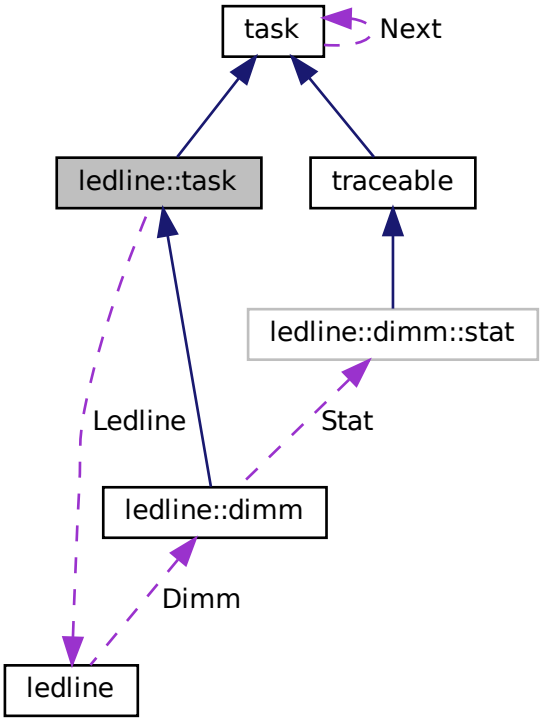
- Ledline.ino

5.11 ledline::task Class Reference

Inheritance diagram for ledline::task:



Collaboration diagram for ledline::task:



Public Member Functions

- **task** (ledline &_ledline)

Protected Attributes

- ledline & **Ledline**

5.11.1 Detailed Description

Ledline driver task. Task with a reference to specific ledline.

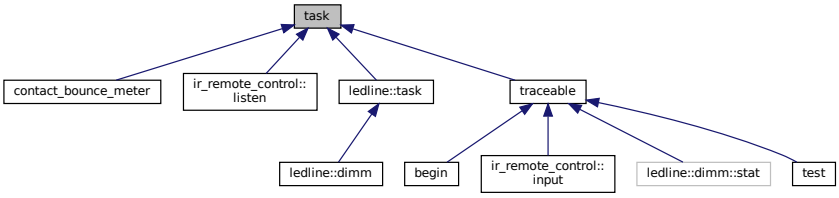
The documentation for this class was generated from the following file:

- Ledline.ino

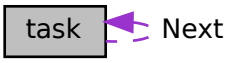
5.12 task Class Reference

```
#include <task.h>
```

Inheritance diagram for task:



Collaboration diagram for task:



Classes

- class queue
- class scheduler

Public Member Functions

- virtual void operator() ()=0

Private Attributes

- task * **Next**

5.12.1 Detailed Description

Task.

5.12.2 Member Function Documentation

5.12.2.1 operator()()

```
virtual void task::operator() ( )  [pure virtual]
```

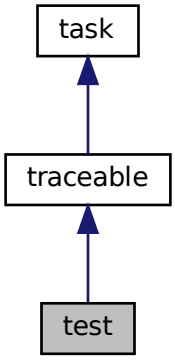
Implemented in contact_bounce_meter, and ir_remote_control::listen.

The documentation for this class was generated from the following file:

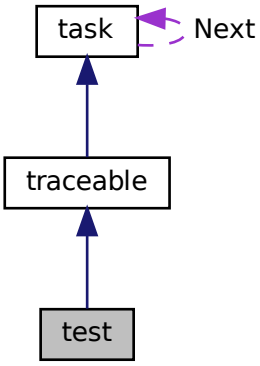
- task.h

5.13 test Class Reference

Inheritance diagram for test:



Collaboration diagram for test:



Static Public Member Functions

- static void **invokeISR** ()

Private Member Functions

- void trace () noexcept

Additional Inherited Members

5.13.1 Detailed Description

Test task.

Test task is used for different testing purposes. It should be modified for each specific case.

5.13.2 Member Function Documentation

5.13.2.1 trace()

```
void test::trace ( )  [inline], [private], [virtual], [noexcept]
```

Defines traceable routine.

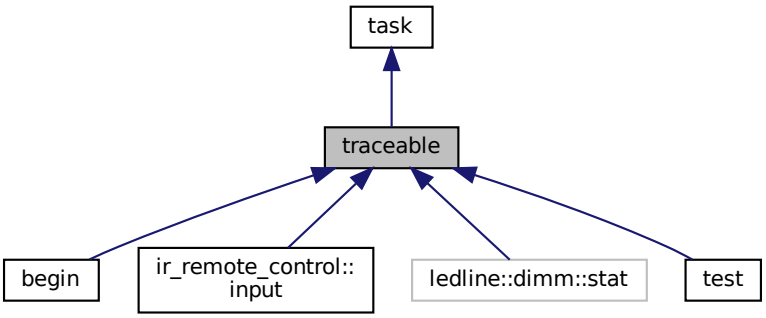
Implements traceable.

The documentation for this class was generated from the following file:

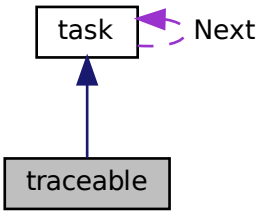
- Ledline.ino

5.14 traceable Class Reference

Inheritance diagram for traceable:



Collaboration diagram for traceable:



Public Member Functions

- virtual void trace () noexcept=0

Private Member Functions

- void operator() ()

5.14.1 Detailed Description

Task which can trace information into serial port.

5.14.2 Member Function Documentation

5.14.2.1 operator()()

```
void traceable::operator() ( ) [inline], [private], [virtual]
```

Implements task.

5.14.2.2 trace()

```
virtual void traceable::trace ( ) [pure virtual], [noexcept]
```

Defines traceable routine.

Implemented in ledline::dimm::stat, ir_remote_control::input, test, and begin.

The documentation for this class was generated from the following file:

- Ledline.ino

5.15 udiv8_t Struct Reference

Public Member Functions

- **udiv8_t** (uint8_t _q, uint8_t _r)

Public Attributes

- uint8_t **quot**
- uint8_t **rem**

The documentation for this struct was generated from the following file:

- udiv.h

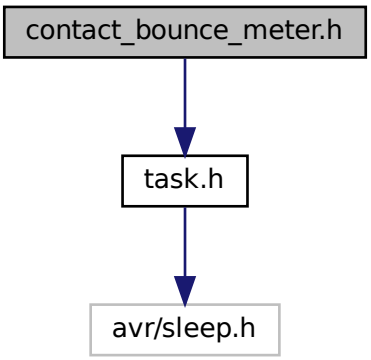
Chapter 6

File Documentation

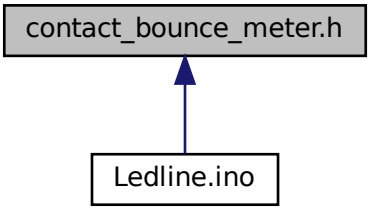
6.1 contact_bounce_meter.h File Reference

```
#include "task.h"
```

Include dependency graph for contact_bounce_meter.h:



This graph shows which files directly or indirectly include this file:



Classes

- class contact_bounce_meter

Functions

- ISR (PCINT0_vect)
- ISR (TIMER1_OVF_vect)
- ISR (TIMER1_CAPT_vect)

6.1.1 Detailed Description

Contact bounce meter measures voltage stabilization time on PB0(PCINT0/CLKO/ICP1).

Used interrupts vectors:

- PCINT0
- TIMER1 OVF
- TIMER1 CAPT

6.1.1.1 Measurements

The following table summarizes measurement results for the different mechanical switches provided by this meter program.

Board column contains measuring board name where

- Nano - Arduino Nano board with 16 MHz Atmega328p MCU.

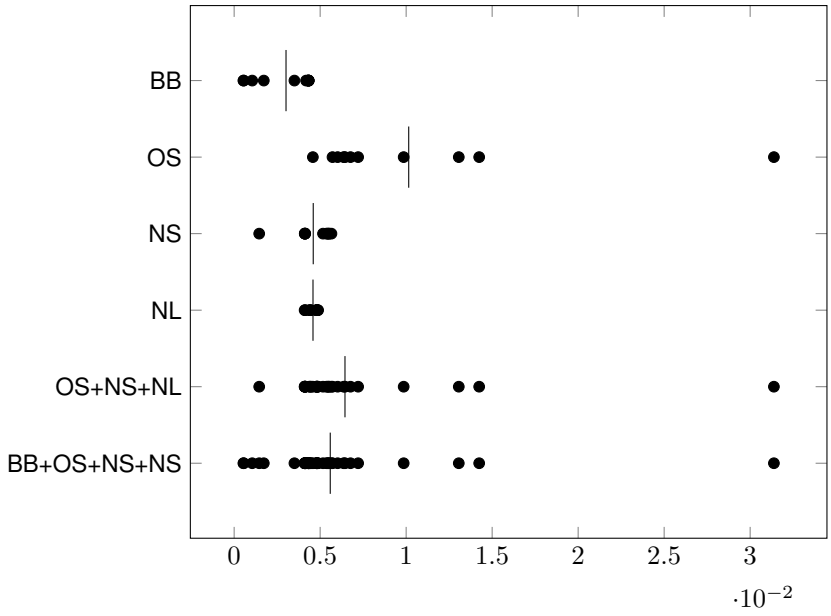
Switch columns contains nominal switch identifier where

- BB - button from arduino set (blue button).
- OS - Old wall switch
- NS - New wall switch
- NL - New wall switch on long wire

Offset column contains number of TCNT1 overflows occured before latest captured value. Capture column contains latest captured TCNT1 value. Overall measured bounce time in timer clocks (Offset$*$0xFFFF+Capture) is in the Time column. Counter column contains number of measuring timer cycles (full cycles from 0 to 0xFFFF). Two last empty sampling cycles stops measurement process. N column contains measurement number for corresponding switch (usually 11 measurement per switch). Column t contains bounce time in seconds calculated according to frequency of MCU clock.

Board	Offset	Capture	Time	Counter	Switch	N	t
Nano	1	3699	69234	4	BB	1	0.0043271
Nano	1	3798	69333	4	BB	2	0.0043333
Nano	0	8924	8924	3	BB	3	0.0005578
Nano	1	1408	66943	3	BB	4	0.0041839
Nano	0	16811	16811	3	BB	5	0.0010507
Nano	1	3812	69347	4	BB	6	0.0043342
Nano	0	56078	56078	3	BB	7	0.0035049
Nano	0	27587	27587	3	BB	8	0.0017242
Nano	1	3463	68998	4	BB	9	0.0043124
Nano	1	3453	68988	4	BB	10	0.0043118
Nano	0	8611	8611	3	BB	11	0.0005382
Nano	1	42724	108259	4	OS	1	0.0067662
Nano	1	30771	96306	4	OS	2	0.0060191
Nano	1	36352	101887	4	OS	3	0.0063679
Nano	1	49801	115336	4	OS	4	0.0072085
Nano	1	37598	103133	4	OS	5	0.0064458
Nano	3	31336	227941	5	OS	6	0.0142463
Nano	2	26641	157711	5	OS	7	0.0098569
Nano	7	43357	502102	9	OS	8	0.0313814
Nano	3	12365	208970	5	OS	9	0.0130606
Nano	1	7693	73228	4	OS	10	0.0045768
Nano	1	25969	91504	4	OS	11	0.0057190
Nano	1	24964	90499	4	NS	1	0.0056562
Nano	1	439	65974	3	NS	2	0.0041234
Nano	1	22928	88463	4	NS	3	0.0055289
Nano	1	590	66125	4	NS	4	0.0041328
Nano	1	22486	88021	4	NS	5	0.0055013
Nano	1	169	65704	4	NS	6	0.0041065
Nano	1	21315	86850	4	NS	7	0.0054281
Nano	1	166	65701	4	NS	8	0.0041063
Nano	1	17127	82662	4	NS	9	0.0051664
Nano	0	23309	23309	3	NS	10	0.0014568
Nano	1	20379	85914	4	NS	11	0.0053696
Nano	1	6098	71633	4	NL	1	0.0044771
Nano	1	11572	77107	4	NL	2	0.0048192
Nano	1	11193	76728	4	NL	3	0.0047955
Nano	1	11391	76926	4	NL	4	0.0048079
Nano	1	11904	77439	4	NL	5	0.0048399
Nano	1	12592	78127	4	NL	6	0.0048829
Nano	1	31	65566	3	NL	7	0.0040979
Nano	1	4465	70000	4	NL	8	0.0043750
Nano	1	644	66179	3	NL	9	0.0041362
Nano	1	5040	70575	4	NL	10	0.0044109
Nano	1	11074	76609	4	NL	11	0.0047881

Visualization of bounce time distribution around its mean value is shown here.



Calculation of some statistics summarized on the following table.

Switch	n	$\bar{t}$	t_{max}	D_c	σ_c	$3\sigma_c$	$\bar{t}+3\sigma_c$
BB	11	0.0030162	0.0043342	0.0000027868	0.0016694	0.0050081	0.0080244
OS	11	0.0101499	0.0313814	0.0000591995	0.0076941	0.0230824	0.0332322
NS	11	0.0045979	0.0056562	0.0000015202	0.0012330	0.0036989	0.0082967
NL	11	0.0045846	0.0048829	0.0000000869	0.0002947	0.0008842	0.0054688
OS+NS+NL	33	0.0064441	0.0313814	0.0000260830	0.0051072	0.0153215	0.0217656
BB+OS+NS+NL	44	0.0055871	0.0313814	0.0000223132	0.0047237	0.0141710	0.0197582

Where

$$\bar{t} = \frac{1}{n}\sum_{i=1}^{n} t_i$$

$$t_{max} = \max_{1\leq i\leq n} t_i$$

$$D_s = \frac{1}{n}\sum_{i=1}^{n}(t_i-\bar{t})^2 = \frac{1}{n}\sum_{i=1}^{n}(t_i^2-2t_i\bar{t}+\bar{t}^2) = \frac{1}{n}\sum_{i=1}^{n}t_i^2 - 2\bar{t}\frac{1}{n}\sum_{i=1}^{n}t_i + \bar{t}^2 = \frac{1}{n}\sum_{i=1}^{n}t_i^2 - 2\bar{t}^2 + \bar{t}^2 = \frac{1}{n}\sum_{i=1}^{n}t_i^2 - \bar{t}^2 \tag{6.1}$$

$$D_c = \frac{1}{n-1}\sum_{i=1}^{n}(t_i-\bar{t})^2 = \frac{1}{n-1}\sum_{i=1}^{n}(t_i^2-2t_i\bar{t}+\bar{t}^2) = \frac{1}{n-1}(\sum_{i=1}^{n}t_i^2 - 2\bar{t}\sum_{i=1}^{n}t_i + n\bar{t}^2) = \frac{n}{n-1}(\frac{1}{n}\sum_{i=1}^{n}t_i^2 - \bar{t}^2) \tag{6.2}$$

$$D_c = \frac{n}{n-1}D_s$$

$$\sigma_c = \sqrt{D_c}$$

Two simpliest circuits will be considered to connect switch to MCU. The first circuit pulls MCU pin down to ground and the second one pulls MCU pin to high voltage when switch is off. If the second circuit is used with builtin

pin pullup resistor then voltage drop exists across resistor R_2 which depends on its resistance (high enough R_2 resitance leads to permanent hight level on pin). That's why all circuits are considered without any external pull resistors (external pull resistor is connected to pin before switch RC-filter).

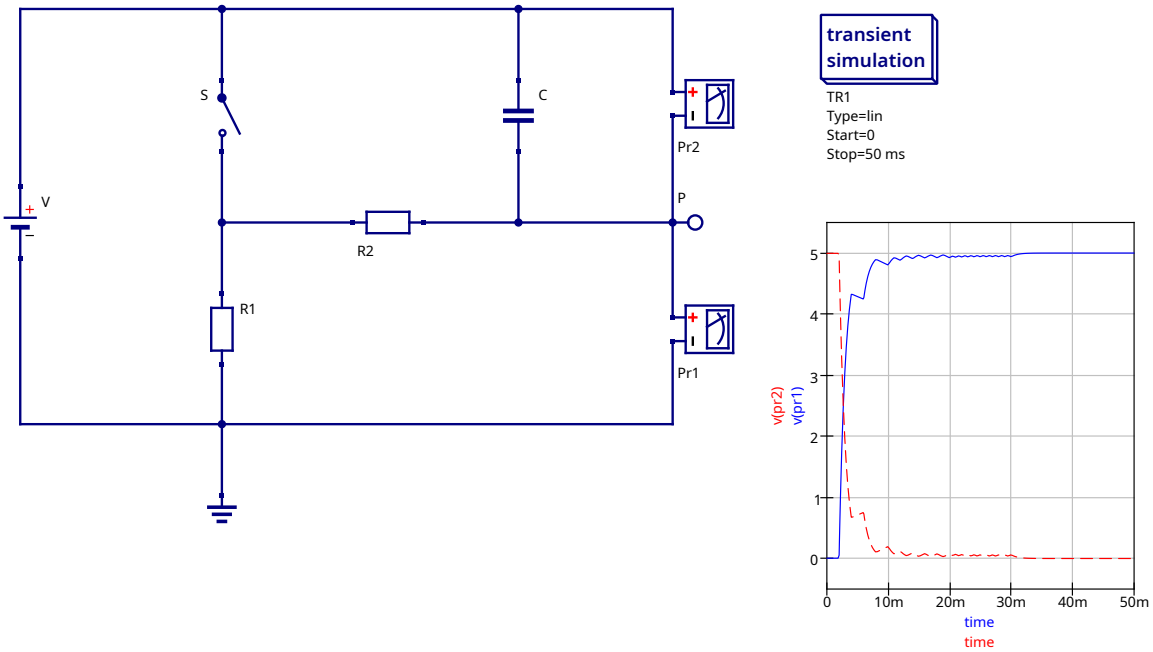


Figure 6.1 Pulldown switch circuit

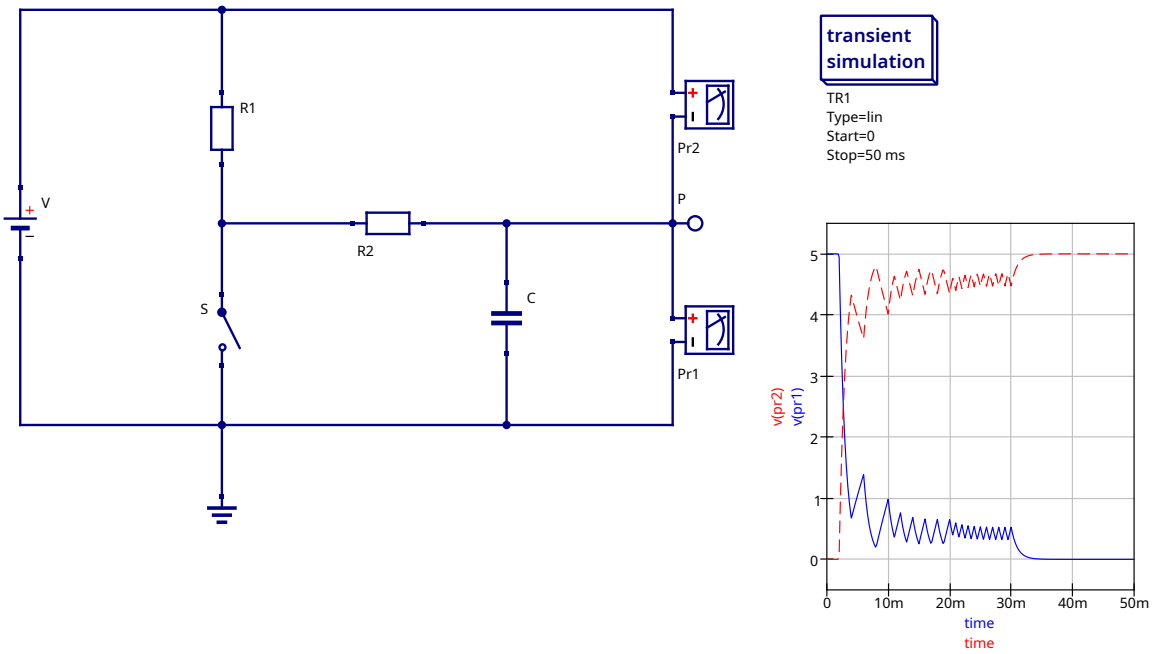


Figure 6.2 Pullup switch circuit

In the case of pull down circuit capacitor C is charging through R_2 resistor and discharging through $R_1 + R_2$ resistors. In the case of pull up circuit capacitor C is charging through $R_1 + R_2$ and discharging through R_2

resistors. So the shortest time of charging/discharging is defined by R_2 resistence and C capacity. Longest time of charging/discharging is defined by $R_1 + R_2$ resistence and C capacity. Time constant of capacitor (time of 63,8% charging or 38,2% discharging) is used to estimate charging/discharging time.

$$\tau_s = CR_2$$

$$\tau_l = C(R_1 + R_2)$$

τ_s should by large enough to cover switch bounce time and in opposite τ_l should be small enough to avoid significant switch delay. R_1 should be chosen to avoid high current consumed by the circuit when switch is closed.

Two main configurations of these circuits will be estimated. The first one is with relatively high resistance and low capacity and the second one is with low resistence and high capacity. In both cases only configurations where $R1 > R2$ will be considered because if $R2 > R1$ the capacitor does not discharge completely (capacitor is a second voltage source competing for the same ground as main voltage source - in the case of higher $R2$ resistance it loses).

Futher calculations are based on maximum estimated bounce time of 0.033 for OS switch (Bounce statistics).

Following table contains calculated by maximum estimated bounce time τ_{max} value $\bar{R_2}$, existing nearest standard resistance value R_2 and corresponding resistance capacitor time constant (τ),

C	F	τ_{max}	R_2	R_2	τ_s
10 nF	0.00000001	0.033	3323223.9	1000000	0.0100
100 nF	0.0000001	0.033	332322.4	220000	0.0220
1 μF	0.000001	0.033	33232.2	10000	0.0100
3.3 μF	0.0000033	0.033	10070.4	10000	0.0330
10 μF	0.00001	0.033	3323.2	2200	0.0220
33 μF	0.000033	0.033	1007.0	1000	0.0330
100 μF	0.0001	0.033	332.3	220	0.0220
220 μF	0.00022	0.033	151.1	220	0.0484
1000 μF	0.001	0.033	33.2	220	0.2200

where $\bar{R_2} = \tau_{max}/F$ and $\tau = FR_2$.

Following combinations of C and R_2 can be used to provide time constant not less than maximim estimated bounce time

C	F	R_2	τ_s
3.3 μF	0.0000033	10000	0.0330
33 μF	0.000033	1000	0.0330
220 μF	0.00022	220	0.0484

For these combinations and R_1 value greater then 10 kOhm (for pull resister) following τ_l are calculted

C	F	R_2	R_1	τ_l
3.3 μF	0.0000033	10000	10000	0.066
3.3 μF	0.0000033	10000	100000	0.363
33 μF	0.000033	1000	10000	0.363
3.3 μF	0.0000033	10000	220000	0.759
3.3 μF	0.0000033	10000	470000	1.584
220 μF	0.00022	220	10000	2.2484
3.3 μF	0.0000033	10000	1000000	3.333
33 μF	0.000033	1000	100000	3.333
33 μF	0.000033	1000	220000	7.293
33 μF	0.000033	1000	470000	15.543
220 μF	0.00022	220	100000	22.0484
33 μF	0.000033	1000	1000000	33.033
220 μF	0.00022	220	220000	48.4484
220 μF	0.00022	220	470000	103.4484
220 μF	0.00022	220	1000000	220.0484

So if $C = 3.3\,\mu F$, $R_1 = 100\,kOhm$ and $R_2 = 10\,kOhm$ are used circuits will provide $\tau_s = 0.033\,s$ and $\tau_l = 0.363\,s$.

6.1.2 Function Documentation

6.1.2.1 ISR() [1/3]

```
ISR (
            PCINT0_vect  )
```

Pin Change Interrupt on ICP1 to start measure.

All other PCINT of PCI0 except ICP1 should be masked to avoid unintended start.

6.1.2.2 ISR() [2/3]

```
ISR (
            TIMER1_CAPT_vect  )
```

Input Capture Interript ISR.

6.1.2.3 ISR() [3/3]

```
ISR (
            TIMER1_OVF_vect  )
```

TOV1 ISR.

6.2 contact_bounce_meter.h

Go to the documentation of this file.

```
1
387 #include "task.h"
388
390 class contact_bounce_meter:public task
391 {
392   static const uint8_t MaxIdleCounter = 2;
393
395   void operator()()
396   {
397     Serial.begin(9600);
398     Serial.println();
399     Serial.print("Offset:"); Serial.println(Offset);
400     Serial.print("Capture:"); Serial.println(Capture);
401     Serial.print("Time:"); Serial.println(Capture + Offset*0x0000FFFFL);
402     Serial.print("Counter:"); Serial.println(Counter);
403     Serial.end();
404   }
405
407   void enablePCINT0()
408   {
409     PCMSK0 |= _BV(PCINT0);
410   }
411
413   void disablePCINT0()
414   {
415     PCMSK0 &= ~_BV(PCINT0);
416   }
417
419   void enableTIMER1_OVF_CAPT()
420   {
```

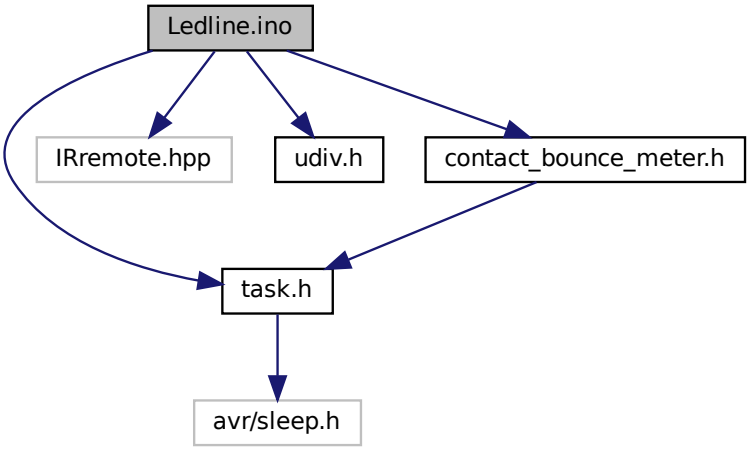
```
    TIMSK1 |= _BV(ICIE1)|_BV(TOIE1);
  }

  void disableTIMER1_OVF_CAPT()
  {
    TIMSK1 &= ~(_BV(ICIE1)|_BV(TOIE1));
  }

  void clearTCNT1()
  {
    TCNT1 = 0;
  }

  uint16_t  Capture;
  uint8_t   Offset;
  uint8_t   Counter;
  bool      Idle;
  uint8_t   IdleCounter;

public:

  static contact_bounce_meter Self;

  contact_bounce_meter()
  {
  }

  void begin()
  {
    PCICR |= _BV(PCIE0);    // Enable PCIE0 interrupts.

    TCCR1A = 0x00;      // Timer operatin mode normal (non-PWM).
    TCCR1B = _BV(CS10);     // Timer prescaler mode - no prescaler, falling edge to trigger input capture.
    TCCR1C = 0x00;      // No compare mode forced.

    enablePCINT0();     // enable PCINT0
  }

  void PCINT()
  {
    Counter = Offset = 0;
    Capture = 0;
    Idle = true;
    IdleCounter = MaxIdleCounter;

    disablePCINT0();
    enableTIMER1_OVF_CAPT();
    clearTCNT1();
  }

  void OVF()
  {
    Counter++;
    if(Idle && --IdleCounter == 0)
      {
    disableTIMER1_OVF_CAPT();
    enablePCINT0();
    task::scheduler::Self.putISR(this);
      }
    Idle = true;
  }

  void CAPT()
  {
    Capture = ICR1;
    Offset = Counter;
    Idle = false;
    IdleCounter = MaxIdleCounter;
  }

};

contact_bounce_meter contact_bounce_meter::Self;

ISR(PCINT0_vect)
{
  contact_bounce_meter::Self.PCINT();
}

ISR(TIMER1_OVF_vect)
{
  contact_bounce_meter::Self.OVF();
}

ISR(TIMER1_CAPT_vect)
{
```

```
  contact_bounce_meter::Self.CAPT();
}
```

6.3 Ledline.ino File Reference

```
#include "task.h"
#include <IRremote.hpp>
#include "udiv.h"
#include "contact_bounce_meter.h"
```

Include dependency graph for Ledline.ino:



Classes

- class traceable
- class ledline
- class ledline::task
- class ledline::dimm
- struct ledline::dimm::stat
- class ir_remote_control
- class ir_remote_control::input
- class ir_remote_control::listen
- class test
- class begin

Macros

- #define **VERSION** 0x0106
- #define **SLEEP** true
- #define **CAR_ADDRESS** 0x0000

- #define **CAR_BUTTON_CH_PREV** 0x45
- #define **CAR_BUTTON_CH** 0x46
- #define **CAR_BUTTON_CH_NEXT** 0x47
- #define **CAR_BUTTON_PREV** 0x44
- #define **CAR_BUTTON_NEXT** 0x40
- #define **CAR_BUTTON_PLAY** 0x43
- #define **CAR_BUTTON_MINUS** 0x7
- #define **CAR_BUTTON_PLUS** 0x15
- #define **CAR_BUTTON_EQ** 0x9
- #define **CAR_BUTTON_0** 0x16
- #define **CAR_BUTTON_100_PLUS** 0x19
- #define **CAR_BUTTON_200_PLUS** 0xD
- #define **CAR_BUTTON_1** 0xC
- #define **CAR_BUTTON_2** 0x18
- #define **CAR_BUTTON_3** 0x5E
- #define **CAR_BUTTON_4** 0x8
- #define **CAR_BUTTON_5** 0x1C
- #define **CAR_BUTTON_6** 0x5A
- #define **CAR_BUTTON_7** 0x42
- #define **CAR_BUTTON_8** 0x52
- #define **CAR_BUTTON_9** 0x4A
- #define **NEB_ADDRESS** 0xFD07
- #define **NEB_BUTTON_ONOFF** 0xA
- #define **NEB_BUTTON_SCAFF** 0xB
- #define **NEB_BUTTON_NEBULA** 0x3
- #define **NEB_BUTTON_B_PLUS** 0x1
- #define **NEB_BUTTON_B_MINUS** 0x5
- #define **NEB_BUTTON_S_PLUS** 0x4
- #define **NEB_BUTTON_S_MINUS** 0x2
- #define **NEB_BUTTON_LA_MINUS** 0xC
- #define **NEB_BUTTON_LA_PLUS** 0xD
- #define **NEB_BUTTON_LA_MODE** 0xE
- #define **NEB_BUTTON_LA_ONOFF** 0xF
- #define **NEB_BUTTON_RESET** 0X1A
- #define **POLARIS_BUTTON_ON_OFF** 0x3B2
- #define **PIN_CONTROL** 3

  PD3(PCINT19/OC2B/INT1)
- #define **PIN_LEDLINE** 5

  PD5(PCINT21/OC0B/T1)
- #define PIN_BOUNCE 8

  PB0(PCINT0/CLKO/ICP1)
- #define PIN_TOGGLE 6

  Toggle pin.
- #define **PIN_RXD** 0

  PD0(RXD/PCINT16)

Functions

- void **loop** ()
- void **setup** ()
- ISR (PCINT2_vect)

Variables

- class ledline **Ledline**

6.3.1 Detailed Description

0.1 Switch on/off with dimming effect. 1.0 Brightness adjustment (NEC:0 only) 1.1 Exponential brightness adjustment, constant speed. 1.2 Shortened brightness transition time (1024 ms). 4 brightness transition levels. Integral (integer) gamma correction. Toggle maximizes brightness. 1.3 Task scheduler with normal and ISR queue. Interrupt to start IrReceiver decoding loop. 1.4 Toggle button task started by PCINT (no polling). Idle mode when schedulers queues are empty. 8 brightness levels . 1.5 Pull up RXD to avoid noise when USB disconnected. Serial port is connected only in tracing tasks. IR receiving process is stoped until pin interrupt. Plus and Minus adjust brightness by 1. 4 preset brightness levels with Next/Prev buttons 1.6 Change gloal task Scheduler to task::scheduler::Self Plus and Minus adjust brightness by 1 after gamma correction.

6.3.2 Macro Definition Documentation

6.3.2.1 PIN_BOUNCE

```
#define PIN_BOUNCE 8
```

PB0(PCINT0/CLKO/ICP1)

Input Capture Pin

Pin to measure contact bounce delay.

6.3.2.2 PIN_TOGGLE

```
#define PIN_TOGGLE 6
```

Toggle pin.

Toggle ledline between to states based on pin change interrupt (PCINTx).

Todo Connect simple one-way switch instead of test button to toggle light to pin. On changing switch state ledline is toggling between min and max levels. It may be used by usual wall light switch to toggle ledline without remote control.

PD6(PCINT22/OC0A/AIN0)

6.3.3 Function Documentation

6.3.3.1 ISR()

```
ISR (
            PCINT2_vect  )
```

Test button interrupt processing.

This interrupt is used for starting different testing routines. Test task should be modified for each specific test (PIN_TOGGLE).

Todo This interrupt is to be used for toggling light from external switch.

6.4 task.h

```
#ifndef __TASK_H__
#define __TASK_H__

#include <avr/sleep.h>

class task
{
public:

    task():Next(NULL){}

    virtual void operator()() = 0;

    class queue
    {
      public:

        queue():Tail(NULL){}

        task* head()
        {
          return Tail;
        }

        task* tail()
        {
          return Tail->Next;
        }

        task* put(task *t)
        {
          if(t->Next == NULL)
          {
            if(Tail)
            {
              t->Next = Tail->Next;
              Tail->Next = t;
            }
            else
              t->Next = t;
            Tail = t;
            return t;     // t is scheduled for execution
          }
          else
            return NULL;  // t is already in a queue and is not scheduled
        }

        task* get()
        {
          auto t = Tail->Next;
          if(t == Tail)
            Tail = NULL;
          else
            Tail->Next = t->Next;
          t->Next = NULL;
          return t;
        }

      private:

        /*
*  Task queue is a circular linked list.
```

```
*              Tail
*                |
*                v
*  T1->T2->...->Tn
*  ^              |
*  +-------------+
*/
        task *Tail;

    };

    class scheduler
    {
    public:

    scheduler():ISRFlag(false){}

        task* put(task* t)
        {
           return Queue.put(t);
        }

        task* putISR(task* t)
        {
          t = ISRQueue.put(t);
          if(t)
            ISRFlag = true;
          return t;
        }

      void execute(bool _sleep = false, uint8_t _mode = SLEEP_MODE_IDLE)
      {
    if(ISRFlag)
      {
        (*ISRQueue.tail())();
        noInterrupts();
        ISRQueue.get();
        if(ISRQueue.head() == NULL)
          ISRFlag = false;
        interrupts();
      }
    else
      if(Queue.head())
        (*Queue.get())();
      else
        if(_sleep)
          {
        noInterrupts();
        if(ISRFlag)
          /* ISRQueue contains a task to process.  Continue processing.  */
          interrupts();
        else
          {
            /* ISRQueue really empty.  Go idle.  */
            set_sleep_mode(_mode);
            sleep_enable();
            interrupts();
            sleep_cpu();
            sleep_disable();
          }
          }
      }

    private:
    queue Queue;
        queue ISRQueue;
        bool  ISRFlag;

    public:

      static scheduler Self;

    };

  private:

    task *Next;

};

#endif
```

6.5 udiv.h

```
struct udiv8_t
{
  uint8_t quot;
  uint8_t rem;
  udiv8_t(){}
  udiv8_t(uint8_t _q, uint8_t _r):quot(_q), rem(_r){}
};

struct udiv8_t udiv(uint16_t _n, uint8_t _d)
{
  return udiv8_t(_n/_d, _n%_d);
}
```

Index

begin, 9
    contact_bounce_meter, 12
    trace, 10
Bound
    ledline, 22
brightness
    ledline, 21

CAPT
    contact_bounce_meter, 13
clearTCNT1
    contact_bounce_meter, 13
contact_bounce_meter, 11
    begin, 12
    CAPT, 13
    clearTCNT1, 13
    contact_bounce_meter, 12
    disablePCINT0, 13
    disableTIMER1_OVF_CAPT, 13
    enablePCINT0, 13
    enableTIMER1_OVF_CAPT, 13
    operator(), 14
    OVF, 14
    PCINT, 14
    Self, 14
contact_bounce_meter.h, 45
    ISR, 51

Day
    ledline, 22
disablePCINT0
    contact_bounce_meter, 13
disableTIMER1_OVF_CAPT
    contact_bounce_meter, 13

enablePCINT0
    contact_bounce_meter, 13
enableTIMER1_OVF_CAPT
    contact_bounce_meter, 13
execute
    task::scheduler, 36

gamma
    ledline, 22
get
    task::queue, 34

ir_remote_control, 19
ir_remote_control::input, 18
    trace, 19
ir_remote_control::listen, 32
    ISRoutine, 33
    operator(), 33
ISR
    contact_bounce_meter.h, 51
    Ledline.ino, 55
ISRoutine
    ir_remote_control::listen, 33

ledline, 20
    Bound, 22
    brightness, 21
    Day, 22
    gamma, 22
    mount, 30
    Night, 22
    No, 22
    operator uint8_t, 30
    operator++, 30
    operator--, 31
    operator=, 31
    operator[], 31
    preset, 21
    Twilight, 22
Ledline.ino, 53
    ISR, 55
    PIN_BOUNCE, 55
    PIN_TOGGLE, 55
ledline::dimm, 15
    next, 17
    operator(), 17
    prev, 17
    set, 17
ledline::dimm::stat, 37
    print, 38
    trace, 38
ledline::task, 38

mount
    ledline, 30

next
    ledline::dimm, 17
Night
    ledline, 22
No
    ledline, 22

operator uint8_t
    ledline, 30
operator()

contact_bounce_meter, 14
ir_remote_control::listen, 33
ledline::dimm, 17
task, 40
traceable, 44
operator++
ledline, 30
operator--
ledline, 31
operator=
ledline, 31
operator[]
ledline, 31
OVF
contact_bounce_meter, 14

PCINT
contact_bounce_meter, 14
PIN_BOUNCE
Ledline.ino, 55
PIN_TOGGLE
Ledline.ino, 55
preset
ledline, 21
prev
ledline::dimm, 17
print
ledline::dimm::stat, 38
put
task::queue, 34

queue
task::queue, 34

Self
contact_bounce_meter, 14
task::scheduler, 36
set
ledline::dimm, 17

task, 40
operator(), 40
task::queue, 33
get, 34
put, 34
queue, 34
task::scheduler, 35
execute, 36
Self, 36
test, 41
trace, 42
trace
begin, 10
ir_remote_control::input, 19
ledline::dimm::stat, 38
test, 42
traceable, 44
traceable, 43
operator(), 44
trace, 44
Twilight
ledline, 22

udiv8_t, 44